Does Visual Page Filtering Help a Text-Only RAG Pipeline? Evaluating ColPali in a Cost-Constrained Biomedical Setting

1. Introduction

Biomedical knowledge is increasingly concentrated in full-text scientific articles whose critical findings are often embedded in tables, figures, and structured layouts rather than in running prose alone. Clinicians, researchers, and automated systems that need to extract factual answers from this literature face a fundamental challenge: the documents they must search are visually rich, but the tools they use to search them are almost entirely text-based.

Retrieval-Augmented Generation (RAG) has become a widely adopted paradigm for question answering over domain-specific document collections. In a typical RAG pipeline, a PDF parser or Optical Character Recognition (OCR) engine extracts text from document pages, a chunking strategy segments that text into retrievable passages, an embedding model or lexical index makes those passages searchable, and a large language model (LLM) generates answers conditioned on the retrieved context. This multi-stage approach has shown strong results across domains including legal, financial, and biomedical information retrieval. However, each stage in this ingestion chain introduces potential failure points. OCR engines may misread complex layouts, chunking heuristics may split tables or captions from the text they describe, and the resulting passages may omit visual information entirely. As Faysse et al. demonstrate, optimizing the ingestion pipeline often yields greater retrieval improvements than optimizing the embedding model itself, underscoring how brittle and consequential this preprocessing chain can be [1].

To address this visual information bottleneck, Faysse et al. propose ColPali, a vision-language model that performs document retrieval by directly embedding images of document pages rather than their extracted text [1]. ColPali extends PaliGemma-3B, a vision-language model that aligns image and text in a shared embedding space, and applies a late interaction mechanism inspired by ColBERT [9] to compute fine-grained similarity between individual query tokens and document image patches. This design enables ColPali to leverage visual cues (tables, figures, layout structure, and typographic emphasis) that text-only retrieval pipelines cannot access. On the Visual Document Retrieval Benchmark (ViDoRe), ColPali achieves an average nDCG@5 of 81.3, outperforming all standard text-based pipelines including those augmented with OCR and figure captioning, with particularly strong results on the healthcare subset (94.4 nDCG@5) [1]. However, the ColPali paper evaluates the model exclusively as a standalone page-level retriever and does not assess whether improved retrieval translates to better downstream answer

generation. As the authors note, combining visual retrieval with answer generation to create end-to-end RAG systems remains a key future direction [1].

Our project takes a step in that direction. We evaluate a practical question that a resource-constrained team might face: **given a text-only LLM, a standard OCR tool, and a fixed-window chunking strategy, is adding ColPali as a visual page pre-filter a worthwhile investment**? We compare two pipelines on biomedical factoid question answering using expert-curated questions from the BioASQ challenge, a well-established benchmark in which domain experts author questions and link them to source articles indexed in PubMed [2], with full-text PDFs retrieved from PubMed Central's Open Access subset. Pipeline A is a lexical baseline: PaddleOCR extracts text from all pages offline, the text is chunked into fixed-window passages, and BM25 [3], a classical term-frequency-based ranking function, retrieves the top-K chunks from the full corpus. Pipeline B adds ColPali as a pre-filtering step: at query time, ColPali identifies the most relevant pages from its offline page embeddings, and BM25 searches only the chunks that overlap those pages. Both pipelines feed their retrieved context to the same text-only LLM for answer generation.

This hybrid approach (using ColPali as a layout-aware page ranker to narrow the search space for a traditional text-based retrieval pipeline) represents a practical middle ground that the ColPali paper does not evaluate. Rather than replacing the text pipeline entirely, Pipeline B uses ColPali only to reduce the candidate set before BM25 re-ranks within it. This lets us isolate whether ColPali's visual understanding adds value even when all downstream components (OCR, chunking, and the LLM) remain entirely text-based. Our evaluation measures both retrieval quality (Hit Rate, Recall) and end-to-end answer quality (Exact Match, ROUGE-L, BERTScore), along with per-query latency, to determine whether ColPali's retrieval improvements propagate through the full pipeline or whether the bottleneck lies elsewhere.

2. Methods

2.1 Dataset Constructions

We use the BioASQ 13b training dataset [2], a well-established benchmark for biomedical question answering. The dataset contains expert-curated questions across four types: factoid, yes/no, list, and summary. We restrict our evaluation to factoid questions, which ask for a specific named entity or short phrase (e.g., "What drug is Entresto?"). Factoid questions admit clean automatic evaluation via Exact Match, where the predicted answer either contains the gold entity or it does not, eliminating the subjectivity involved in evaluating longer free-text responses.

Each BioASQ question is linked to one or more PubMed articles (via PMIDs) that contain the supporting evidence. To obtain full-text PDFs for our retrieval pipeline, we resolved PMIDs to

PMC identifiers using the NCBI ID Converter API and filtered to articles available as Open Access PDFs in PubMed Central. We computed an OA ratio for each question, defined as the fraction of its gold documents available as downloadable PDFs, and selected questions with an OA ratio of at least 0.5. We accumulated unique documents until reaching approximately 120 target articles, then included any additional question whose gold documents were already covered by this corpus at no extra download cost. This two-pass selection maximizes question coverage while keeping the document corpus manageable.

The final dataset contains 69 factoid QA pairs linked to 102 unique biomedical PDFs comprising 1,017 pages (mean 10.0 pages per document, range 2 to 23). On average, each question is linked to 1.5 gold source documents, and the corpus contains 2,216 text chunks after processing. Because all documents are drawn from the biomedical literature, the corpus contains substantial terminological overlap across articles (shared gene names, drug names, and disease terms), which makes retrieval more challenging by requiring the system to discriminate between documents that discuss similar topics. Furthermore, a 69-factoid QA pair might seem like a small benchmark set, but the number that matters more here is the 1000+ pages corpus we are working with. Given the 5-hour OCR and 12-hour ColPali preprocessing times, we believe this QA pair set size sufficiently represents our dataset, while remaining manageable on our individual devices.

2.2 Pipeline Architecture Overview

We compare two pipelines that share all offline preprocessing, chunking, LLM, and prompt design, isolating ColPali's contribution as the sole experimental variable. Both pipelines operate under a fixed context budget of K=5 retrieved chunks, ensuring the LLM receives the same amount of context in both conditions.

Pipeline A (Lexical Baseline). All 1,017 pages are OCR'd offline and chunked into 2,216 text passages stored in SQLite. At query time, BM25 scores every chunk in the corpus against the query, and the top-5 chunks by score are assembled into a prompt and sent to the LLM for answer generation.

Pipeline B (Visual-Guided Hybrid). The same OCR text and chunks are produced offline, and ColPali page embeddings are additionally computed for all 1,017 pages and stored in Qdrant. At query time, the query is first embedded with ColPali and used to retrieve the top-P most relevant pages from the full corpus of 1,017 pages via MaxSim search. The chunk pool is then filtered to only those chunks whose page ranges overlap the retrieved pages, and BM25 scores this reduced candidate set. The top-5 chunks from this filtered pool are assembled into the same prompt template and sent to the same LLM. We evaluate two settings, P=5 and P=10, to assess how the aggressiveness of the page pre-filter affects both retrieval quality and downstream answer quality.

This design ensures that any difference in output is attributable solely to the retrieval step. The same OCR engine, chunking parameters, BM25 implementation, prompt template, and LLM are used in both pipelines. The only difference is that Pipeline B narrows the BM25 search space from all 2,216 chunks to the subset overlapping ColPali's top-P pages before ranking.

Stage	Pipeline A (Lexical Baseline)	Pipeline B (Visual-Guided Hybrid)
Offline Preprocessing		
PDF Rendering	300 DPI per-page PNGs	Same
OCR	PaddleOCR v3 (mobile)	Same
Chunking	400-word windows, 100-word overlap → 2,216 chunks in SQLite	Same
ColPali Embeddings	--	ColPali v1.3 embeds all 1,017 pages → Qdrant
BM25 Index	BM25Okapi over all 2,216 chunks	Same
Query-Time Retrieval		
Step 1	BM25 scores all 2,216 chunks	ColPali embeds query, retrieves top-P pages via MaxSim (P=5, P=10)
Step 2	Select top-5 chunks	Filter chunks to retrieved pages, BM25 scores candidates
Step 3	--	Select top-5 chunks from filtered set
Generation		
LLM	Llama 3.1 8B via Ollama (K=5 chunks)	Same
Prompt	Biomedical factoid QA template	Same

Table 1: Side-by-side pipeline comparison showing each stage (Offline Preprocessing, Query-Time Retrieval, Generation) for Pipeline A vs Pipeline B.

2.3 Offline Processing

PDF Rendering. Each PDF is converted to per-page PNG images at 300 DPI using the pdf2image library. This resolution preserves fine detail in tables and figures while remaining computationally tractable for OCR processing.

OCR. We use PaddleOCR v3 [5] with the PP-OCRv5 mobile detection model for text extraction. PaddleOCR is a layout-aware OCR engine that detects text regions within a page, recognizes their content, and sorts them in reading order (top-to-bottom, left-to-right) using bounding polygon coordinates. We selected PaddleOCR for its strong reported accuracy on clean digital PDF renders and its efficient inference speed. The mobile detection model is sufficient for our 300 DPI images and avoids the higher memory and latency costs of server-grade models.

Chunking. OCR text from each document is concatenated in page order and segmented into fixed-window chunks of 400 words with 100-word overlap. We keep the text extraction pipeline constant across both pipelines, ensuring that any performance difference is attributable to ColPali's page pre-filtering rather than to differences in chunking strategies. Another intention behind this chunking strategy, as opposed to semantic or section-aware splitting, is to assess the efficacy of an easy-to-maintain, developer-friendly method while staying true to our cost-constrained setup. A short-tail merge rule ensures the final chunk contributes at least 50 new words; otherwise it is merged into the preceding chunk. Each chunk records its source document (PMCID) and the range of pages it spans, enabling the page-to-chunk mapping that Pipeline B requires. All 2,216 chunks are stored in SQLite with a global index that maps directly to BM25's positional scoring array.

ColPali Embeddings (Pipeline B only). We use ColPali v1.3 (vidore/colpali-v1.3-hf) [1], a PaliGemma-3B extension trained for document retrieval. ColPali generates multi-vector representations from page images, producing one 128-dimensional embedding vector per image patch. Queries are similarly encoded into per-token vectors. Retrieval uses a late interaction mechanism (inspired by ColBERT [9], which retains individual token-level embeddings rather than pooling them into a single vector) where each query token is matched to its highest-scoring image patch via a maximum similarity (MaxSim) operation, and the token-level scores are summed to produce a page-level relevance score. All 1,017 page embeddings are stored in Qdrant, a vector database configured for multi-vector cosine similarity with the MaxSim comparator. Embedding is performed once per page as an offline step; at query time, only the query needs to be embedded.

BM25 Index. We build a BM25 index [3] over all 2,216 chunks using the rank-bm25 Python library. BM25 is a keyword-based search algorithm that scores chunks by how often query terms appear in them, how rare those terms are across the corpus, and how long the chunk is. The same text preprocessing is applied to both the document chunks and the incoming query to ensure consistent matching.Once again, staying true to our cost constraints, we chose BM25 over dense

embedding models to establish a transparent, reproducible baseline that doesn't demand a resource-heavy setup. Another merit worth noting: biomedical text is terminology-heavy, with precise terms like drug names, gene symbols, and disease codes that benefit from exact keyword matching, making BM25 a competitive baseline in this domain. The ColPali paper's own results show that within the same ingestion pipeline configuration, the gap between BM25 and BGE-M3 is modest compared to the gains achieved by improving the ingestion pipeline itself [1].

2.4 Online Retrieval and Generation:

Pipeline A Retrieval. The query is tokenized using the same lowercase alphanumeric tokenizer used at index time, and BM25 scores all 2,216 chunks. The top-5 chunks by score are retrieved from SQLite and assembled into the LLM prompt.

Pipeline B Retrieval. The query is first embedded using ColPali's text encoder, producing a multi-vector query representation. This representation is used to search Qdrant for the top-P most relevant pages across the full corpus via MaxSim. The retrieved pages are mapped to chunk global indices through SQLite, using the page-range metadata recorded during chunking in which a chunk is included in the candidate set if its page range overlaps any of the top-P pages. BM25 then scores only these candidate chunks, and the top-5 by score are selected and assembled into the same prompt template.

LLM Generation. Both pipelines use the same prompt template, which presents the retrieved chunks as numbered context passages and instructs the model to answer the question in a few words based only on the provided context, or respond with "unanswerable" if the context is insufficient. The full prompt is structured as:

"You are a biomedical expert. Answer the following question in a few words based only on the provided context. If the context does not contain the answer, say 'unanswerable'. [Context] [Question]"

The LLM is Llama 3.1 8B [4], served locally via Ollama with a fixed random seed (seed=42) to ensure that identical prompts produce identical outputs across runs. This allows us to attribute any differences in answer quality between Pipeline B variants (P=5, P=10) solely to retrieval differences rather than generation variance. Pipeline A processes all 69 questions first, then Pipeline B processes all 69 questions sequentially, rather than interleaving them. This way we force the resetting of the LLM's KV cache, which ensures each pipeline starts with an empty cache, and latency numbers reflect a conservative estimate by eliminating any speed advantages from cached data.

2.5 Evaluation Metrics:

Answer Quality. We use three complementary metrics to capture different aspects of answer correctness. Exact Match (EM) tests whether the predicted answer contains the gold entity: it is scored as 1 if the prediction contains any of the accepted exact answers as a case-insensitive substring, and 0 otherwise. BioASQ questions may have multiple acceptable answers (e.g., both a brand name and a generic name), and a match against any one of them counts as correct. ROUGE-L [8] measures word-level overlap between the prediction and the ideal answer using the longest common subsequence (LCS), which captures word ordering without requiring consecutive matches. We report the F1 score with stemming enabled so that morphological variants (e.g., "approved" and "approving") are treated as matches. BERTScore [7] measures semantic similarity using contextual embeddings from RoBERTa-large, capturing meaning-level equivalences that surface-level overlap misses (e.g., "FTY720" and "Fingolimod" refer to the same drug but share no lexical overlap). For both ROUGE-L and BERTScore, questions with multiple ideal answers are scored against each variant, and the maximum F1 is taken.

Retrieval Quality. Hit Rate@K measures whether at least one gold source document appears among the documents associated with the top-K retrieved chunks, scored as 1 (hit) or 0 (miss). Recall@K measures the fraction of gold source documents found among the top-K retrieved chunks. Both metrics operate at the document level: a retrieved chunk is considered a hit if its source PMCID matches any of the question's gold PMCIDs, regardless of whether that specific chunk contains the answer text. We evaluate at K=1 and K=3 to assess both the top-ranked result and a slightly broader retrieval window. In our dataset, 50 of 69 questions (72%) are linked to a single gold document, for which Hit Rate@K and Recall@K are equivalent. Recall@K provides additional information for the remaining 19 questions with multiple gold documents, where it captures whether the pipeline found all relevant sources rather than just one.

Efficiency. We measure retrieval latency, LLM latency, and total latency per query. Retrieval latency includes all steps from query tokenization through chunk selection (and for Pipeline B, ColPali embedding and Qdrant search). LLM latency measures the time for Ollama to generate the answer. Separating these components allows us to isolate the latency cost that ColPali adds to Pipeline B's retrieval step, while confirming that generation time remains comparable across both pipelines.

ColPali Pre-Filter Analysis (Pipeline B only). Page Hit Rate measures whether ColPali's top-P pages include at least one page from a gold source document. Page Recall measures the fraction of gold documents that appear anywhere in the top-P pages. These metrics assess how effectively ColPali narrows the search space before BM25 re-ranking and provide an upper bound on Pipeline B's retrieval quality. If ColPali fails to surface a gold document's pages, BM25 cannot recover it.

3. Results

3.1 Main Results:

Metric	Pipeline A (BM25)	Pipeline B (P=5)	Pipeline B (P=10)
Answer Quality			
Exact Match	68.1%	66.7%	66.7%
ROUGE-L (F1)	0.262	0.269	0.252
BERTScore (F1)	0.863	0.861	0.860
Retrieval Quality			
Hit Rate@1	91.3%	98.6%	95.7%
Hit Rate@3	95.7%	98.6%	98.6%
Recall@1	0.769	0.825	0.796
Recall@3	0.863	0.873	0.874
Efficiency			
Retrieval Latency (s)	0.01	0.34	0.31
LLM Latency (s)	11.1	11.2	11.2
Total Latency (s)	11.1	11.5	11.5
ColPali Pre-filter			
Page Hit Rate	--	100.0%	100.0%
Page Recall	--	0.960	0.981

Table 2: This presents the full comparison between Pipeline A and both Pipeline B variants across all evaluation metrics. All runs use a fixed random seed for LLM generation, ensuring that answer quality differences are attributable to retrieval changes rather than sampling variance. Pipeline A results are identical across runs, confirming reproducibility.

Retrieval Quality. Both Pipeline B variants consistently outperform Pipeline A on retrieval metrics. At the more aggressive setting (P=5), Hit Rate@1 reaches 98.6% compared to 91.3% for Pipeline A, meaning Pipeline A fails to retrieve a gold document in its top-ranked chunk on 6 of 69 questions while Pipeline B (P=5) fails on only 1. Recall@1 improves from 0.769 to 0.825. At P=10, Hit Rate@1 is 95.7% and Recall@1 is 0.796, both still above Pipeline A but slightly below P=5. Hit Rate@3 is 98.6% for both variants, and Recall@3 is comparable across all conditions (0.863 to 0.874). The ColPali pre-filter achieves a perfect Page Hit Rate of 100% at both P values, meaning every query's candidate pages include at least one page from a gold document. Page Recall increases from 0.960 at P=5 to 0.981 at P=10, indicating that a larger page budget recovers more of the secondary gold documents but does not affect whether the primary gold document is found.

Answer Quality. Despite the retrieval improvements, all three configurations produce nearly identical answer quality. Exact Match is 68.1% for Pipeline A and 66.7% for both Pipeline B variants. ROUGE-L F1 ranges from 0.252 to 0.269, and BERTScore F1 from 0.860 to 0.863, with no consistent advantage for any configuration. The fixed seed confirms that these small differences are not caused by generation randomness but reflect genuine (though negligible) variation in the retrieved context. We examine the Exact Match gap further in Section 3.3.

3.2 Efficiency

ColPali pre-filtering adds approximately 0.3 seconds of retrieval latency per query (0.34s at P=5 and 0.31s at P=10, compared to 0.01s for Pipeline A), reflecting the cost of embedding the query with ColPali and searching Qdrant via MaxSim. This overhead was measured on an Apple M3 Pro with 36 GB of memory using MPS acceleration. The ColPali paper reports approximately 30ms per query on an NVIDIA L4 GPU [1], suggesting that with dedicated GPU hardware the per-query latency gap would be negligible. LLM generation dominates total latency at approximately 11 seconds per query for all configurations, making the retrieval overhead approximately 3% of total query time. Overall, both Pipeline B variants are 0.4 seconds slower per query (11.5s vs 11.1s), a 4% increase.

It is worth noting that ColPali's offline embedding cost (computing page embeddings for all 1,017 pages) is a one-time preprocessing step. Once the embeddings are stored in Qdrant, Pipeline B's per-query overhead is limited to the query embedding and vector search. For a stable document corpus that is queried repeatedly, this amortized cost is modest relative to the retrieval quality gains reported in Section 3.1.

3.3 Analysis

With a fixed random seed, Pipeline A achieves an Exact Match of 68.1% and both Pipeline B variants achieve 66.7%, a difference of 1 question (47 vs 46 correct out of 69). At P=5, Pipeline A and Pipeline B agree on 64 of 69 questions: Pipeline A gets 3 correct where Pipeline B does

not, while Pipeline B gets 2 correct where Pipeline A does not. At P=10, the pipelines agree on 68 of 69 questions, with only 1 disagreement where Pipeline A succeeds and Pipeline B does not. The near-identical Exact Match across both P values confirms that the pre-filter aggressiveness has minimal impact on answer quality.

Examining the disagreement cases reveals that differences in retrieved chunks, not generation randomness, now explain the Exact Match gap. For example, for the question "What periodontal disease associated bacteria is also associated with Alzheimer's disease?", Pipeline A retrieves chunks from both gold documents (PMC5575416 and PMC5652018) and generates the correct answer "Porphyromonas gingivalis." Pipeline B at P=5 retrieves chunks exclusively from PMC5652018, and the LLM generates "Treponema denticola," a different bacterium mentioned in the same document. At P=10, Pipeline B retrieves a broader set of chunks spanning both gold documents, and the LLM correctly generates "Porphyromonas gingivalis." This example illustrates how a tighter page pre-filter can occasionally exclude relevant context, though the effect is small.

Comparing Pipeline B across P values, 4 of 69 questions produce different Exact Match outcomes between P=5 and P=10, with 2 favoring each setting. Different page candidate sets lead to different retrieved chunks, which in turn lead to different LLM answers. However, the aggregate Exact Match remains identical (66.7%), confirming that no single P value is systematically better for exact answer match. It is also worth noting, that Exact Match uses case-insensitive substring matching, which is strict. This means, synonyms, abbreviations, and alternative nomenclature score 0 even when semantically correct. These false negatives affect both pipelines equally but explain why the absolute scores (66-68%) understate true accuracy of both pipelines.

Breaking down the per-question results by the 19 questions linked to multiple gold documents, Recall@K reveals a nuance that Hit Rate@K does not.. Both Pipeline B variants achieve a near-perfect Hit Rate@1 on these questions, meaning they always retrieve at least one gold document in their top-ranked chunk. However, Recall@3 on questions with multiple gold documents is comparable across all configurations (0.609 for Pipeline A, 0.591 for B at P=5, 0.596 for B at P=10), indicating that neither pipeline consistently retrieves chunks from all gold documents when multiple are relevant. With a fixed budget of K=5 retrieved chunks, covering multiple source documents is inherently difficult regardless of retrieval strategy. For the 50 questions with a single gold document, Pipeline B outperforms Pipeline A on Hit Rate@1 at both P=5 (0.980 vs 0.920) and P=10 (0.940 vs 0.920).

The stability of ROUGE-L and BERTScore across all configurations, despite Pipeline B's higher retrieval quality, suggests that both pipelines retrieve sufficient context for the LLM to produce semantically comparable answers. When Pipeline A retrieves chunks from non-gold documents, those chunks still contain relevant biomedical terminology from the same domain, providing the

LLM with enough context to approximate the correct answer. The improvement from retrieving the correct pages appears to be marginal when the LLM's answer quality is already constrained by upstream tasks like OCR fidelity and the choice of chunking and retrieval strategy.

The central observation is that ColPali's retrieval improvements do not translate to answer quality improvements. Even when ColPali successfully identifies the correct pages, the downstream text extraction, chunking, and generation steps remain unchanged, and it is these shared stages where information is most likely lost. We discuss the implications of this finding in Section 4.

4. Discussion

4.1 Summary of Findings

This project evaluated whether adding ColPali as a visual page pre-filter to a standard text-based RAG pipeline improves biomedical question answering. We tested two pre-filter settings (P=5 and P=10 pages) against a BM25-only baseline using 69 expert-curated factoid questions from BioASQ and 102 full-text biomedical PDFs from PubMed Central. Our results show that ColPali is highly effective as a page-level retriever, achieving 100% Page Hit Rate and near-perfect Page Recall at both P values, with Hit Rate@1 improving from 91.3% (Pipeline A) to as high as 98.6% (Pipeline B, P=5). However, this retrieval improvement does not translate to improved answer quality: Exact Match, ROUGE-L, and BERTScore remain effectively unchanged across all configurations. The bottleneck lies downstream of retrieval, in the shared OCR, chunking, and generation stages that both pipelines depend on.

4.2 Cost-Constrained Design Philosophy

Our pipeline was designed to reflect a realistic, cost-constrained deployment scenario. Every component was chosen for simplicity and low maintenance overhead. We chose, BM25 for retrieval, fixed-window chunking, a general-purpose OCR engine, and a lightweight text-only LLM. This mirrors the stack a small team might realistically deploy for internal document search over a stable corpus, such as legal contract lookup or regulatory compliance documentation, where upgrading to a vision-language model or layout-aware OCR pipeline is not feasible.

In this context, ColPali represents a minimal addition to the stack. It requires a one-time offline embedding pass over the document corpus and adds approximately 0.3 seconds of per-query latency, roughly 3% of total query time. It requires no GPU at deployment time and no changes to the retrieval or generation stack. The question we set out to answer was whether this single addition is worth the investment. Our results suggest that for retrieval quality, the answer is yes: ColPali reliably surfaces the correct source documents with near-perfect accuracy, and as a side effect enables precise page-level citations that users can consult directly when the LLM answer alone is insufficient. For answer quality, the answer is not yet, because the downstream

components that both pipelines share constrain the ceiling. The binding bottleneck is not which pages are ranked but how faithfully the text is extracted and chunked from them.

4.3 Why ColPali Over Dense Embeddings

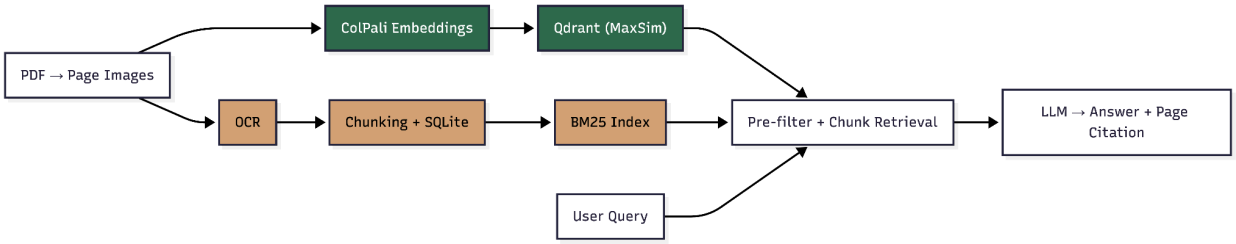

Figure 1: To illustrate the divergence and decoupling of ColPali and OCR pre-processing tasks

A natural question is why we chose ColPali rather than a dense semantic retrieval model such as BGE-M3 as the additional retrieval component. The key distinction is architectural independence. Every text-based retrieval method, whether sparse (BM25) or dense (BGE-M3, ColBERT), operates downstream of OCR. If OCR garbles a table, linearizes a multi-column layout incorrectly, or fragments an equation, every text-based retriever fails identically because they share the same corrupted input. As demonstrated in Figure 1, ColPali operates on a parallel, independent stream directly from the raw page image. Its retrieval quality is decoupled from OCR quality entirely. This means that even when answer quality is bottlenecked by the text extraction pipeline, ColPali's page-level judgments remain trustworthy because they are not derived from the same potentially corrupted text. In our results, ColPali achieved 100% Page Hit Rate across both P values, confirming that it reliably identifies the correct source pages regardless of how well OCR captured their content.

4.4 ColPali as a Citation Layer

Although ColPali did not improve answer quality in our evaluation, its perfect Page Hit Rate demonstrates a distinct practical value: citation and provenance. In a real-world biomedical information tool, users need to verify answers against the original source material. ColPali can reliably point users to the exact PDF page containing the relevant information, independent of whether the text extraction pipeline faithfully captured that page's content. This citation capability has standalone value in domains where trust and verifiability are critical, such as clinical decision support or regulatory document review, even when the generated answer itself does not measurably improve.

4.5 Strengths

Our evaluation offers several methodological strengths. The controlled comparison holds all variables constant except the retrieval strategy: both pipelines use the same OCR engine, the

same chunks, the same BM25 index, the same LLM, and the same prompt template, isolating ColPali's contribution cleanly. We evaluate the full RAG pipeline end-to-end, measuring generated answer quality rather than only retrieval accuracy, which is a gap the ColPali paper explicitly identifies as a key future direction [1]. The hybrid architecture we test, using ColPali as a page ranker feeding into text-based retrieval, represents a practical middle-ground deployment that the ColPali paper does not evaluate. Testing two P values (P=5 and P=10) with a fixed random seed provides a reproducible comparison that isolates retrieval changes from generation variance. Finally, our corpus uses real biomedical PDFs with natural terminological overlap across articles, providing realistic retrieval difficulty without synthetic distractors.

4.6 Limitations

Several limitations should be noted. Our corpus is scoped to 102 biomedical documents comprising 1,017 pages. At this scale, BM25 can exhaustively score all 2,216 chunks in milliseconds, which limits the practical efficiency advantage of pre-filtering. ColPali's value as a candidate-set reducer would likely become more pronounced at larger corpus scales where exhaustive scoring becomes a bottleneck. PaddleOCR's mobile detection model is a general-purpose OCR engine not optimized for the complex layouts found in scientific papers, including dense tables, nested figures, chemical formulas, and multi-column text. A specialized scientific document parser could provide substantially cleaner text for both pipelines. Our evaluation uses BioASQ factoid questions, which ask for short named entities that are typically found in the text body of articles. Although the biomedical PDFs themselves contain rich visual elements such as tables and figures, the factoid questions do not require understanding those elements to produce a correct answer. This limits our ability to demonstrate ColPali's advantage on visually complex content. An ideal evaluation dataset would include questions whose answers are embedded in tables, figures, or diagrams, but no such dataset currently exists for the biomedical domain. BioASQ is biomedical but text-oriented, while DocVQA [1] is visually oriented but not biomedical. We also tested a single context budget (K=5 retrieved chunks) and did not evaluate whether increasing K would allow Pipeline B's retrieval improvements to translate into answer quality gains.

4.7 Future Work

Several directions could extend this work. Replacing PaddleOCR with a specialized scientific document parser such as Nougat [6], which is trained specifically on arXiv and PMC papers to output structured markup, could raise the text quality ceiling and potentially allow ColPali's retrieval advantages to propagate to answer quality. A direct multimodal approach would skip OCR entirely and pass ColPali's retrieved page images to a vision-capable LLM such as GPT-4V or Gemini, eliminating the text extraction bottleneck altogether. Applying ColBERT-style late interaction to OCR text rather than page images would help isolate whether ColPali's retrieval advantage comes from the late interaction architecture itself or specifically from visual

understanding. Testing on larger corpora where BM25 exhaustive scoring becomes a meaningful bottleneck would better demonstrate ColPali's efficiency value. Evaluating on question types beyond factoid, particularly list and summary questions from BioASQ that require synthesizing information across multiple documents and visual elements, would provide a more complete picture of when visual pre-filtering matters. Varying the context budget K would determine whether Pipeline B benefits from a larger retrieval window. Constructing or identifying a mixed dataset with both text-answerable and visually-answerable biomedical questions remains an open challenge that would enable a more definitive evaluation of visual retrieval in this domain.

5. Conclusion

Our results provide empirical evidence for the direction the ColPali authors propose, which is combining visual retrieval with answer generation to create end-to-end RAG systems [1]. In our text-only pipeline, ColPali's retrieval improvements are absorbed by the shared downstream bottleneck of OCR and chunking. However, ColPali already delivers practical value as an OCR-independent page ranking and citation layer. For teams operating under cost constraints with text-only LLMs, ColPali is a worthwhile addition for retrieval quality and source attribution, even before the ecosystem evolves toward fully vision-native RAG architectures.

Works Cited

[1] M. Faysse, H. Sibille, T. Wu, B. Omrani, G. Viaud, C. Hudelot, and P. Colombo, "ColPali: Efficient Document Retrieval with Vision Language Models," in Proc. International Conference on Learning Representations (ICLR), 2025.

[2] A. Nentidis, G. Katsimpras, A. Krithara, and G. Paliouras, "Overview of BioASQ Tasks 13b and Synergy13 in CLEF2025," in CLEF 2025 Working Notes, 2025.

[3] S. Robertson, S. Walker, S. Jones, M. Hancock-Beaulieu, and M. Gatford, "Okapi at TREC-3," in Proceedings of the Third Text REtrieval Conference (TREC), 1994.

[4] A. Dubey et al., "The Llama 3 Herd of Models," arXiv:2407.21783, Meta AI, 2024.

[5] PaddlePaddle Team, "PaddleOCR 3.0 Technical Report," arXiv:2507.05595, 2025.

[6] L. Blecher, G. Cucurull, T. Scialom, and R. Stojnic, "Nougat: Neural Optical Understanding for Academic Documents," arXiv:2308.13418, 2023.

[7] T. Zhang, V. Kishore, F. Wu, K. Weinberger, and Y. Artzi, "BERTScore: Evaluating Text Generation with BERT," in Proc. ICLR, 2020.

[8] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," in Text Summarization Branches Out, ACL, 2004.

[9] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," in Proc. SIGIR, 2020.